

Introduction to the course and development tools

CS 101 - Introduction to Programming II

Table of Contents

- [1. Course overview](#)
- [2. From C++ To C and Back Again: Instructor Introduction](#)
- [3. Abstractions: The key to our schedule](#)
- [4. Confusing: C++ sample codes](#)
- [5. Difficult: C++ and its Standards](#)
- [6. Truth: God is in the detail](#)
- [7. Advice: How should you study?](#)
- [8. Advice: How you could study using AI](#)
- [9. Objective: From CS consumer to CS producer](#)
- [10. Syllabus \(walk-through\)](#)
- [11. Summary \(the important stuff\)](#)
- [12. NEXT C++ Review](#)

1. Course overview



The syllabus is available on [GitHub](#).

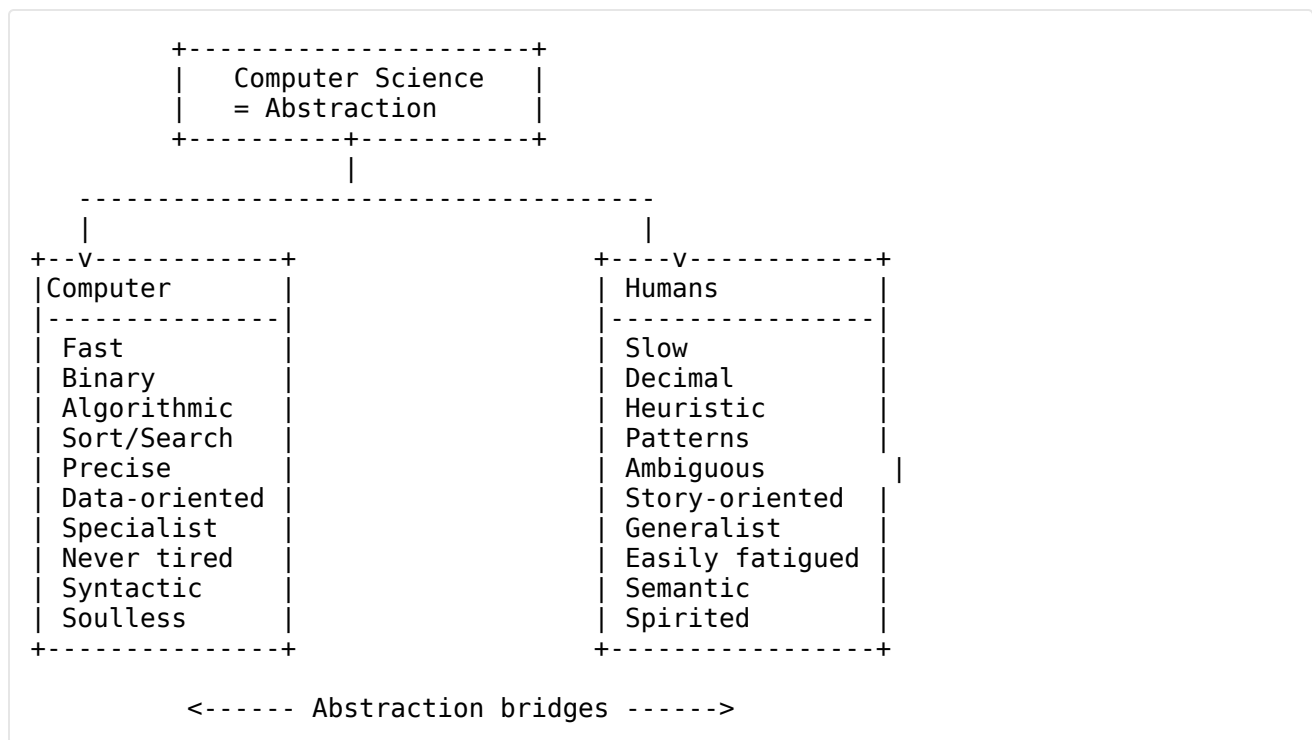
2. From C++ To C and Back Again: Instructor Introduction

- Background and experience
- Course objectives and expectations

3. Abstractions: The key to our schedule

- CS is a science of **abstractions** that are used to bridge the gap between man and machine.

Question: How do computers differ from humans?



- How do "abstractions" bridge the gap between Man and Machine?

Computer	Abstraction bridging gap	Humans
Fast	Batch jobs, schedulers manage pacing	Slow
Binary	Number systems, text encodings	Decimal
Algorithmic	Heuristics encoded as algorithms	Heuristic
Sort/Search	Pattern recognition via ML models	Patterns
Precise	Error bars, tolerances in software	Ambiguous
Data-oriented	Visualization, narratives in reports	Story-oriented
Specialist	General-purpose languages, OS layers	Generalist
Never tired	Automation of repetitive workflows	Easily fatigued
Syntactic	Semantics via programming languages	Semantic
Soulless	HCI, design, artful interfaces	Spirited

- What is the limit of abstractions?

Abstractions are like a translation layer between two profoundly alien entities, one of whom (machines) are made by the other (man). They cannot turn machines into men or men into machines.

Abstractions are also never loss-less: By sacrificing detail, and by creating a machine representation of reality, aspects of reality are suppressed.

- How does this impact our course on introductory programming II?

The course has several attractors or main themes:

1. Understanding abstract (user-defined) data types.
2. Understanding memory control.
3. Understanding object-orientation.
4. Understanding containers and iterators.

Course topics	Abstractions
Overview, infrastructure C++ review	
Intro to Data Structures	Abstract Data Types
C-style Arrays	Collections
Introduction to STL Vectors	Extensible collections
Time & Space Complexity	Complexity
Pointers and arrays	Memory control
Dynamical memory allocation	Memory control
Classes and UML	Object-orientation
STL Vector class	Sequential containers
STL Map class	Associative containers
Linked lists (opt)	Container adapters
Stacks (opt)	Container adapters
Queues (opt)	Container adapters
Recursion (opt)	Problem solving
Trees (opt)	Graph theory
Hash Tables + Graphs (opt)	Graph theory

4. Confusing: C++ sample codes

Can you guess what this code does?

1. Arrays, auto type, range-based loops: weeks 1-4

```
double temps[] = {22.5, 25.0, 19.5, 21.0, 23.5, 26.1};
for (auto temp : temps)
    cout << temp << "C = "
        << temp * 9./5. + 32. << "F." << endl;
```

2. Smart pointers, pointer arithmetic, dynamical memory allocation: weeks 5-7

```
auto str = std::make_unique<char[]>(6);
std::strcpy(str.get(), "Hello");
char* ptr = str.get();
while (*ptr != '\0')
    ptr++;
std::cout << "Length = " << (ptr - str.get()) << "\n";
```

3. Classes, vectors, maps, STL: weeks 8-10

```
map<int,string> emp =
    {{101,"Jim Jones"} , {102,"Adam Miller"},
     {103,"Beth Young"}, {104, "Jake Brown"}};
emp.erase(103);
for (map<int,string>::iterator it = emp.begin();
     it != emp.end();
     it++)
    cout << it->first << ", " << it->second << endl;
```

4. Linked lists, stacks, queues: weeks 11-12 (perhaps)

```
std::stack<int> st;
st.push(10);
st.push(20);
st.pop();
```

5. Recursion, trees, hash tables, graphs: weeks 13-15 (perhaps)

```
void inorder(Node* r) {
    if (!r) return;
    inorder(r->L);
    std::cout << r->key << " ";
    inorder(r->R);
}
```

Answer:

1. Iterates through an array of temperatures and prints Celsius–Fahrenheit conversions.
2. Walks through a string until the null terminator is found and prints its length.
3. Uses a map to store employee records, erase one, and print the rest.
4. Demonstrates pushing and popping values on a stack (LIFO).
5. Recursively traverses a binary tree in inorder sequence.

5. Difficult: C++ and its Standards

It's complicated.



Figure 1: Overlapping C++ standards

To learn data structures, we will stick to a mix of C++11 and C++17.

Photo: Bjarne Stroustrup - Every monitor here probably runs a different C++ standard...in German: "Was man nicht im Kopf hat, muss man in den Beinen haben." (If you don't use your head, you'll have to use your feet.)



Version	Year	Key Features
C++98	1998	First ISO standard; templates, STL, exceptions
C++03	2003	Bug fixes and minor improvements to C++98
C++11	2011	auto, range-based for, lambdas, move semantics, nullptr, smart pointers
C++14	2014	Generic lambdas, make_unique, relaxed constexpr, variable templates
C++17	2017	if constexpr, structured bindings, optional, variant, string_view, filesystem
C++20	2020	Concepts, ranges, coroutines, modules, span, chrono cal, more constexpr
C++23	2023	More constexpr, std::expected, multidim subscript, improved ranges/lambdas
C++26	2026	Executors, reflection, pattern matching, library enhancements (TBD)

6. Truth: God is in the detail



- Paying attention to small details leads to excellence in computing.
- You're now getting immersed in a world of excruciating technical details:
 1. **Excruciating** because unlike in mathematics there isn't a theory or a clean, memorable hierarchy of axioms to hold on to.

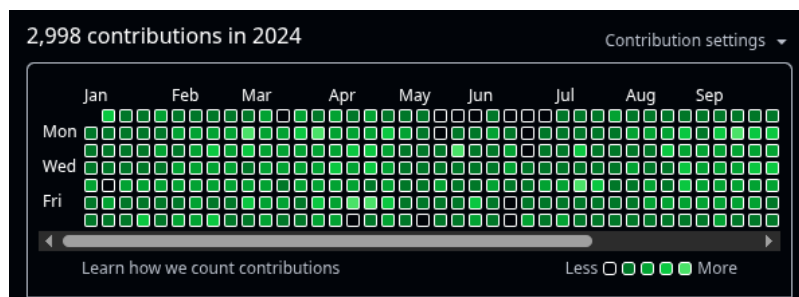
2. **Technical** (from the Greek word τεχνικός, "skillful" or "artful") because controlling machines depends on precision and tools that are learnt by practice rather than deduction.
3. **Detailed** because every small piece matters: a missing semicolon, a wrong bracket, a single mistyped character can bring the whole structure down.

7. Advice: How should you study?

- **Short list** from my [new FAQ \(2024\) on GitHub](#):
 1. Code every day no matter how little, in any language.
 2. Review lecture notes and notebooks before and after class.
 3. Participate actively in the class. Ask questions!
 4. Complete assignments well before the deadline. Ask questions!
 5. Practice 'iterate programming' by documenting your process.

This is what you do when you use Jupyter notebook

6. Focus on fundamentals, simple examples, solid understanding.
7. Drill yourself using the weekly quizzes, make small examples.
8. Seek help when needed, on the chat or arrange a meeting with me.
9. Build a code & notebook repository at GitHub for your resume.



10. Solve problems of increasing difficulty on open.kattis.com

8. Advice: How you could study using AI

- For what it's worth: I fed this part to ChatGPT and asked about changes because of Generative AI (GAI) - I mostly agree except **try not to use AI at all** because it is extremely hard for a **beginner** to know when he should stop trying. **Rule of thumb:** For any topic, allot twice the time, for any new topic, thrice the time you think you need.
 1. **Lectures:** Attend all lectures as before—presence still matters. Use GAI to:
 - Clarify unclear definitions or steps after class.
 - Rephrase explanations in simpler terms or alternative perspectives.
 - Simulate follow-up Q&A on difficult parts of the lecture.
 2. **Private study:** Study actively, not passively—GAI is a tool, not a shortcut. Use GAI to:
 - Generate hints for exercises without revealing solutions.
 - Ask conceptual questions in your own words.
 - Test your understanding by explaining it to GAI and checking its paraphrase.
 3. **Problem-Solving:** Try every problem independently first. Use GAI when stuck to:

- Request a single hint or next step.
- Debug your reasoning or identify common mistakes.
- Generate similar practice problems to reinforce concepts.
- 4. **Tutorials:** Use GAI to simulate a mini-tutorial beforehand:
 - Practice explaining your solutions out loud to the AI.
 - Ask GAI to critique clarity or suggest alternate reasoning.
- 5. **Vacation and Review Work:** Use GAI to:
 - Generate custom quizzes or flashcards.
 - Review old material interactively.
 - Preview future course content through exploratory questioning.
- 6. **General Principle**
 - Always try first—then ask AI.
 - Use GAI to deepen understanding, not to shortcut thinking.
 - Combine traditional study skills with AI tools for best results.

9. Objective: From CS consumer to CS producer

I found this in the preface to "An Infinite Descent into Pure Mathematics" (free book at infinitedescent.xyz) by Clive Newstead.

»Consumers of mathematics are expected to say how they used their tools to find their answers. Producers of mathematics, on the other hand, have to do much more: they must be able to keep track of definitions and hypotheses, piece together facts in new and interesting ways, and make their own definitions of mathematical concepts. But even more importantly, once they have done this, they must communicate their findings in a way that others find intelligible, and they must convince others that what they have done is correct, appropriate and worthwhile.«

Adapted to computer science:

1. **Consumers of computer science** can say how they used their tools to find their answers to computational problems.
2. **Producers of computer science:**
 - Keep track of specifications and assumptions.
 - Combine algorithms and data structures in new & interesting ways.
 - Design their own abstract data types and systems.
 - Communicate their solutions in understandable ways.
 - Convince others that what they have built is correct, appropriate and worthwhile.

10. Syllabus (walk-through)

1. General course information [meeting details, textbook]
2. Objectives [ADTs, memory, OOP, containers, iterators]
3. Target audiences [attitude & readiness]
4. Student learning outcomes [get ready for more - CS 201]
5. Course requirements [Intro to programming, college algebra]
6. GitHub [Materials], Canvas [Assignments, quizzes], Teams [class]
7. Weekly schedule [program, test every week]
8. Grading [programming + practice + tests + optional final exam]
9. Rubric [100% to 0%]
10. About [not] Using AI

11. Summary (the important stuff)

- Computer science is about abstractions that are used to bridge the gap between man and machine.
- Abstractions represent the real world (like a President who represents his people).
- Only paying attention to detail leads to excellence in computing.
- You should code every day no matter how little.
- Your total study time should be consistent with full time work.
- You should study without AI until you know what you're doing.

12. **NEXT** C++ Review

You may only have had Java in the last class as your only/first language. This section of the course will prepare you for C++.

- Process:
 1. We go through the elements of all high level languages: variables, types, conditionals, loops, arrays, and functions.
 2. I will contrast Java and C++ (and a little Python at the start), and you code along with me.
 3. After each concept, you solve a small programming problem in C++.
 4. Your first longer home programming exercise will be to complete a similar set of simple programming problems in C++.
 5. Throughout the course, the emphasis will be on doing, and you should spend most of your time outside of class coding.

Author: Marcus Birkenkrahe

Created: 2026-02-21 Sat 18:33